

Projeto C111

Aluno-1: Guilherme Felipe Ribeiro - GEC - 2042

Aluno-2: Pedro Tiso Vinhas Mesquita – GEC - 1932

Aluno-3: Roger Pereira Freitas – GES - 443

Aluno-4: Jose Carlos Rebouças Neto – GES – 647

<https://github.com/Gpatinho/Projeto-C111>

1. Introdução

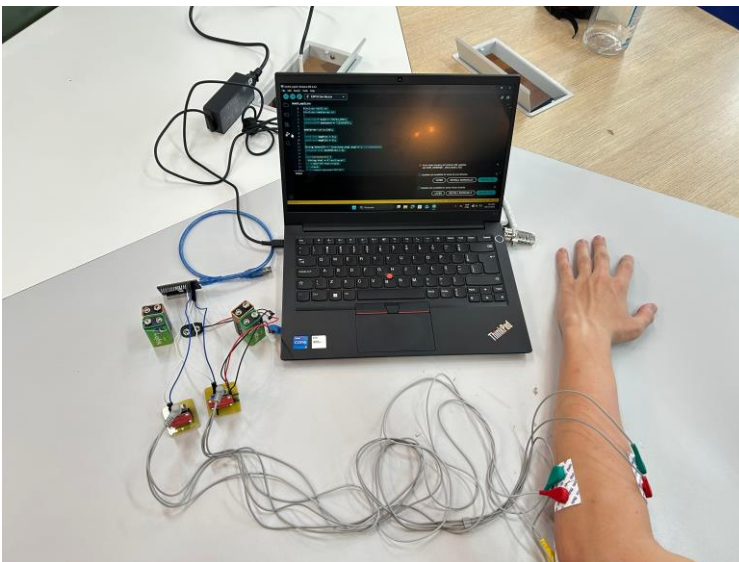
Tema: Análise de Sinais Mioelétricos (EMG) para Reconhecimento e Diferenciação de Movimentos Musculares

Justificativa:

O uso de sensores EMG representa uma solução acessível e eficaz para o desenvolvimento de sistemas de controle de próteses, uma vez que permite a detecção direta da atividade elétrica gerada pelos músculos. A análise desses sinais é essencial para identificar padrões de contração muscular, possibilitando o reconhecimento dos movimentos assim possibilitando futuramente uma incrementação para uma possível controle de uma prótese.

Descrição dos datasets:

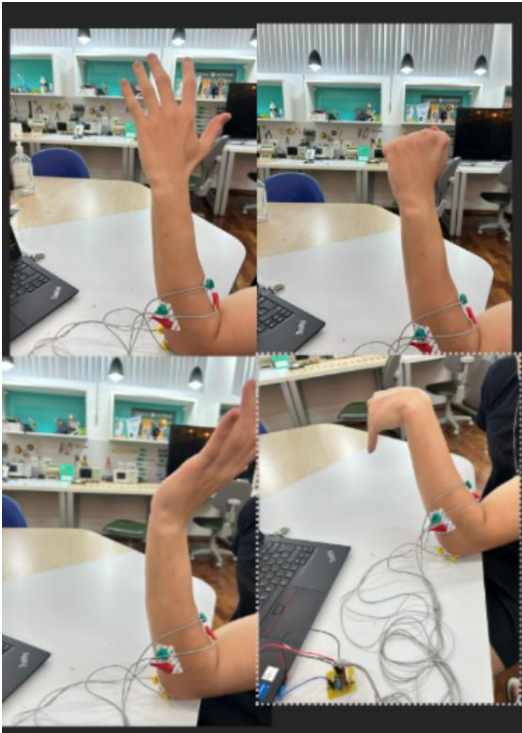
Os datasets utilizados neste trabalho são compostos por capturas reais de sinais elétricos musculares (EMG), adquiridos por meio de dois sensores conectados a um microcontrolador **ESP32**. Cada arquivo CSV contém valores de tensão (mV) em função do tempo (ms), distribuídos em dois canais correspondentes aos dois sensores EMG, acoplados através de eletrodos ECG descartáveis.



A coleta dos sinais foi realizada seguindo uma metodologia controlada e reproduzível. Inicialmente, o membro foi mantido em repouso total, garantindo que os sinais registrados representassem o nível basal de atividade muscular, estabelecendo assim uma linha de referência limpa e livre de interferências provenientes de movimentos.

Posteriormente, foram induzidas contrações musculares específicas para obtenção dos dados referentes aos movimentos de interesse: **flexão do punho, extensão dos dedos e fechamento da mão**. Para cada movimento, o músculo correspondente foi estimulado por aproximadamente 2 segundos, assegurando tempo suficiente para a correta captura da atividade elétrica. Em

seguida, o músculo foi liberado por mais 2 segundos, permitindo o retorno ao estado de repouso e garantindo a distinção clara entre os períodos de contração e relaxamento no sinal adquirido.



Essa abordagem sistemática permitiu gerar um dataset consistente, padronizado e adequado para análises posteriores, incluindo filtragem, processamento de sinais e aplicação de técnicas de reconhecimento de padrões.

Para cada movimento registrado, o dataset contém uma coluna para os dados do **Sensor 1**, outra para os dados do **Sensor 2**, além da coluna de **tempo (ms)**. Os valores presentes não seguem uma sequência numérica contínua; trata-se de múltiplas amostras variando ao longo do tempo, com diferentes magnitudes de acordo com a atividade elétrica captada pelos sensores. O resultado é uma tabela extensa composta por diversos valores inteiros e variáveis, refletindo a dinâmica real dos sinais musculares.

Objetivo:

Processar, filtrar e analisar os sinais EMG coletados, de modo a identificar padrões característicos capazes de distinguir diferentes movimentos musculares,

2. Preparação e limpeza dos dados

Problemas encontrados:

1- Saturação do sinal: Estamos utilizando o microcontrolador **ESP32** para realizar a aquisição dos sinais elétricos musculares. No entanto, o ESP32 apresenta algumas limitações relacionadas à amplitude e ao tempo de amostragem, o que impacta diretamente a qualidade dos dados coletados para a construção do dataset. Esse fator levou ao primeiro desafio identificado no projeto.

Durante a análise inicial dos dados, observamos a presença significativa de **ruídos elétricos**. Ao plotar os sinais brutos — sem aplicar qualquer técnica de filtragem — ficou evidente um nível elevado de interferência, resultando em saturação e distorção dos valores capturados.

Para mitigar esses problemas e melhorar a qualidade do dataset, implementamos um **divisor de tensão** no circuito de aquisição. Essa abordagem permite reduzir a amplitude do sinal de entrada, evitando a saturação do conversor analógico-digital (ADC) do ESP32 durante a coleta dos sinais musculares do antebraço, garantindo assim dados mais consistentes e adequados para processamento e análise.

Gráfico extensão_1 sem divisor de tensão:

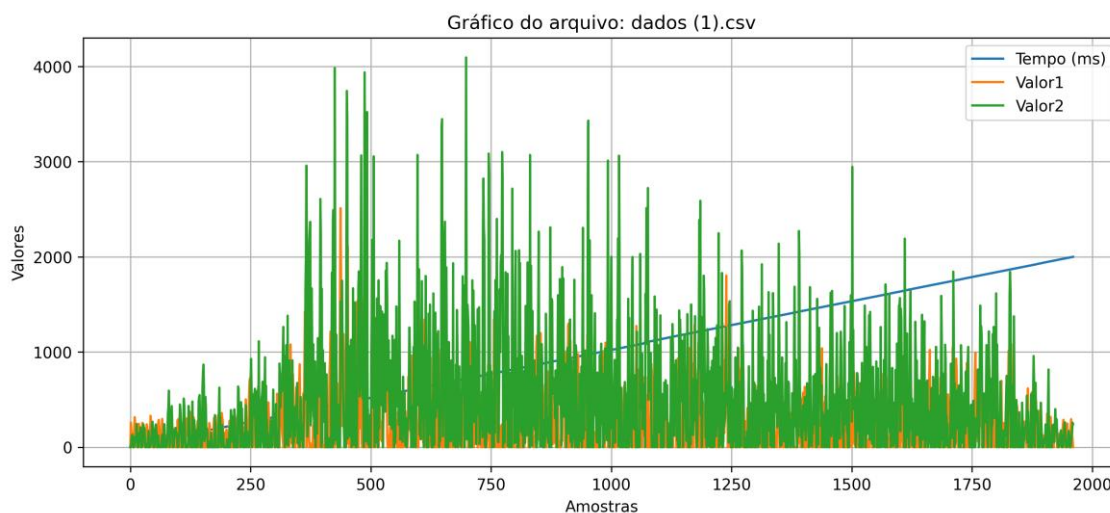
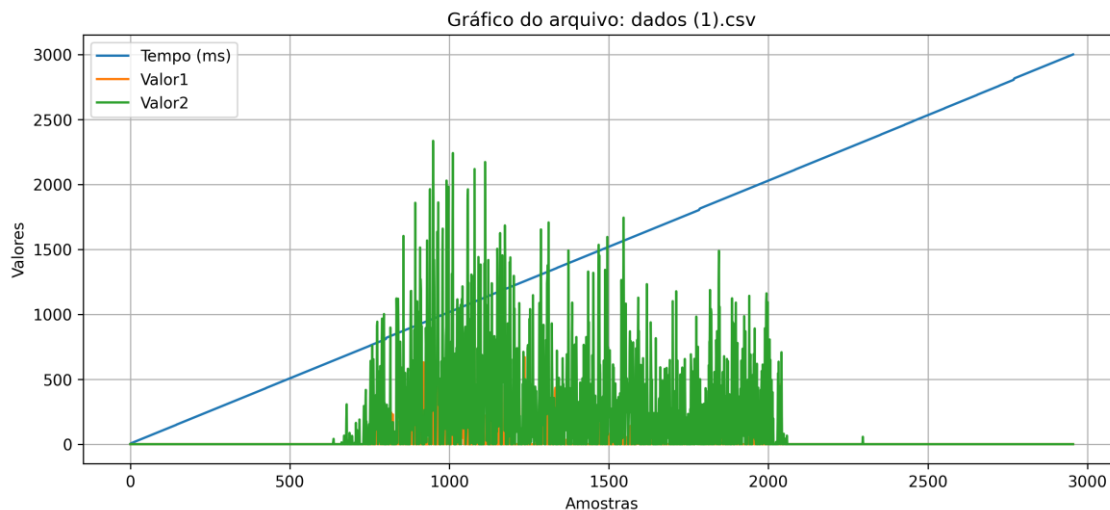


Gráfico extensão_1 com divisor de tensão:



Código usado para plotar os gráficos dos primeiros dataset:

```
import pandas as pd
import matplotlib.pyplot as plt
from google.colab import files
import os
```

```
# ---- 3. Processamento e plotagem ----
for filename in uploaded.keys():
    print(f"\n🚀 Lendo arquivo: {filename}")

    # Carregar CSV
    df = pd.read_csv(filename)

    # Mostrar primeiras linhas
    print(df.head())

    # Plot automático
    plt.figure(figsize=(12,5))
    for col in df.columns:
        if df[col].dtype != 'object': # Só plota colunas numéricas
            plt.plot(df[col], label=col)

    plt.title(f"Gráfico do arquivo: {filename}")
    plt.xlabel("Amostras")
    plt.ylabel("Valores")
    plt.legend()
    plt.grid(True)
```

Explicando partes do código :

df = pd.read_csv(filename)

Lê o conteúdo do CSV

Converte tudo para um DataFrame do pandas.

plt.figure(figsize=(12,5))

Abre um espaço gráfico com tamanho 12x5 polegadas
ainda não tem nada plotado, apenas a "tela".

for col in df.columns:

```
if df[col].dtype != 'object': # Só plota colunas numéricas
```

```
plt.plot(df[col], label=col)
```

Esse bloco faz um trabalho importante:

Ele percorre todas as colunas do DataFrame

Ele verifica se a coluna é numérica (int/float)

Colunas com texto (como "Tempo (ms)" string) são ignoradas

Plota cada coluna como uma linha no gráfico

Cada linha recebe uma legenda com o nome da coluna

isso torna o código universal, pois ele funciona mesmo que as colunas mudem de nome.

```
plt.title(f"Gráfico do arquivo: {filename}")
```

```
plt.xlabel("Amostras")
```

```
plt.ylabel("Valores")
```

```
plt.legend()
```

```
plt.grid(True)
```

Essas linhas fazem o polimento final do gráfico

2- Irregularidade na amostragem temporal

Os intervalos entre amostras não são uniformes ao longo do tempo. A coluna de tempo apresenta incrementos irregulares, o que indica que o sistema de coleta não manteve uma taxa de amostragem fixa. Isso dificulta análises no domínio da frequência, comparações temporais e segmentação do sinal.

dataset extensão_1 sem divisor de tensão:

	A	B	C		A	B	C
1	Tempo (ms)	Valor1	Valor2	781	800	782	710
2	3	0	0	782	801	0	1839
3	5	259	0	783	802	0	934
4	7	176	71	784	803	188	293
5	8	146	110	785	804	187	559
6	9	48	131	786	805	630	1822
7	10	0	99	787	806	368	12
8	11	0	0	788	807	0	1168
9	12	25	0	789	808	0	254
10	13	194	0	790	809	59	0
11	14	315	68	791	810	350	625
12	15	252	112	792	811	539	779
13	16	173	112	793	812	163	68
14	17	81	112	794	813	197	0
15	18	0	243	795	814	563	0
16	19	0	54	796	815	0	2717
17	20	20	0	797	816	112	1022

dataset extensão_1 com divisor de tensão:

	A	B	C		A	B	C
1	Tempo (m)	Valor1	Valor2		781	787	0
2	3	0	0		782	788	0
3	5	0	0		783	789	37
4	7	0	0		784	790	90
5	8	0	0		785	791	250
6	9	0	0		786	792	0
7	10	0	0		787	793	8
8	11	0	0		788	794	16
9	12	0	0		789	795	0
10	13	0	0		790	796	0
11	14	0	0		791	797	0
12	15	0	0		792	798	0
13	16	0	0		793	799	0
14	17	0	0		794	800	0
15	18	0	0		795	801	0
16	19	0	0		796	802	0
					797	803	0
					798	804	0
					799	805	0

Transformações aplicadas para filtragem:

Media móvel: A **média móvel** é um filtro digital simples e eficiente utilizado para **suavizar sinais**, reduzir ruídos e tornar a forma do sinal mais estável. Ele funciona substituindo cada ponto do sinal pela média dos pontos vizinhos dentro de uma “janela” definida.

$$y[n] = \frac{1}{N} \sum_{k=0}^{N-1} x[n - k]$$

x[n] = sinal original

y[n] = sinal filtrado

N = tamanho da janela

k = índice das amostras anteriores

Uma **janela** define quantos valores são considerados para calcular a média.

Exemplo com janela = 5:

Janela de 5 amostras:

[x1, x2, x3, x4, x5] → média

[x2, x3, x4, x5, x6] → média

...

Isso significa que o filtro analisa sempre os últimos 5 valores coletados.

Como o filtro suaviza o sinal?

O ruído em sinais EMG geralmente aparece como:

oscilações rápidas.

picos isolados.

variações bruscas.

Quando você aplica a média móvel:

picos isolados são reduzidos.

transições ficam mais suaves.

tendência geral do sinal fica mais clara. Isso ocorre porque o filtro "espalha" a influência de cada ponto entre os vizinhos.

Aplicamos filtros de média móvel com janelas de 5, 10 e 20 amostras para suavizar o sinal EMG.

O filtro funciona substituindo cada ponto pelo valor médio de seus vizinhos, reduzindo picos e ruídos.

Janelas menores preservam mais detalhes (mais sensibilidade), enquanto janelas maiores geram maior suavização (menos ruído).

Essa filtragem permite observar padrões de ativação muscular com maior clareza.

```
import pandas as pd
import numpy as np

def aplicar_filtro_media_movel(arquivo_entrada, janela=5):
    # Carregar os dados do arquivo CSV
    dados = pd.read_csv(arquivo_entrada)

    # Aplicar filtro de média móvel para cada coluna de valores
    dados['Valor1_filtrado'] = dados['Valor1'].rolling(window=janela, center=True, min_periods=1).mean()
    dados['Valor2_filtrado'] = dados['Valor2'].rolling(window=janela, center=True, min_periods=1).mean()

    # Criar novo DataFrame com as colunas filtradas
    dados_filtrados = pd.DataFrame({
        'Tempo (ms)': dados['Tempo (ms)'],
        'Valor1_filtrado': dados['Valor1_filtrado'],
        'Valor2_filtrado': dados['Valor2_filtrado']
    })
```

Apartir do modelo acima conseguimos fazer algumas mudanças e ir alterando até achar o melhor formato de gráfico.

```
# Função para filtrar e salvar
def aplicar_filtro_e_salvar(nome_arquivo, janela):
    # Carrega CSV
    dados = pd.read_csv(nome_arquivo)

    # Checa nome das colunas
    if "Tempo (ms)" in dados.columns:
        tempo_col = "Tempo (ms)"
    elif "Tempo" in dados.columns:
        tempo_col = "Tempo"
    else:
        raise KeyError("Coluna de tempo não encontrada (esperado: 'Tempo (ms)' ou 'Tempo')")

    # Aplica filtro de média móvel
    dados['Valor1_filtrado'] = dados['Valor1'].rolling(window=janela, center=True, min_periods=1).mean()
    dados['Valor2_filtrado'] = dados['Valor2'].rolling(window=janela, center=True, min_periods=1).mean()
```

Explicando partes do código :

Define uma função chamada `aplicar_filtro_e_salvar`.

Ela recebe dois parâmetros:

- `nome_arquivo`: o nome do arquivo CSV a ser processado
- `janela`: tamanho da janela do filtro de média móvel (ex: 5, 10, 20)

Checa nomes das colunas:

- "Tempo (ms)"
- ou "Tempo"

Se a coluna existir, salva o nome em `tempo_col`.

Se nenhuma for encontrada, o código **gera um erro**, informando que não existe coluna de tempo com os nomes esperados.

filtro de média móvel nas colunas `Valor1` e `Valor2`.

Aplicando média móvel:

- `rolling(window=janela)`
cria uma janela deslizante com tamanho `janela` (ex: 5, 10 ou 20 amostras)
- `center=True`
significa que o filtro é centrado no ponto atual, não deslocado para frente ou para trás
→ gera curvas mais alinhadas
- `min_periods=1`
permite calcular mesmo quando não há amostras suficientes no início ou no fim
→ evita valores nulos (NaN)
- `.mean()`
calcula a média dentro da janela
→ isso é o filtro de média móvel propriamente dito

O resultado é uma versão **suavizada** de cada sinal:

- Valor1_filtrado
- Valor2_filtrado

Essas colunas passam a existir no DataFrame dados.

Após a aplicação dos filtros de **média móvel** nos datasets, a visualização dos sinais torna-se significativamente mais clara e adequada para análise, permitindo identificar padrões e características relevantes de forma mais precisa.

```
# Plota gráfico com escala fixa
plt.figure(figsize=(10, 6))

# Sinal 1
plt.subplot(2, 1, 1)
plt.plot(dados[tempo_col], dados['Valor1'], label='Original', alpha=0.4)
plt.plot(dados[tempo_col], dados['Valor1_filtrado'], label=f'Filtrado {janela}', color='blue')
plt.title('Sinal 1')
plt.ylabel('Amplitude')
plt.ylim(0, 4000) # Escala fixa Y
plt.xlim(0, 3000) # Escala fixa X
plt.legend()

# Sinal 2
plt.subplot(2, 1, 2)
plt.plot(dados[tempo_col], dados['Valor2'], label='Original', alpha=0.4)
plt.plot(dados[tempo_col], dados['Valor2_filtrado'], label=f'Filtrado {janela}', color='green')
plt.title('Sinal 2')
plt.xlabel('Tempo (ms)')
plt.ylabel('Amplitude')
plt.ylim(0, 4000) # Escala fixa Y
plt.xlim(0, 3000) # Escala fixa X
plt.legend()

plt.tight_layout()
```

Explicando partes do código :

plt.figure(figsize=(10, 6))

Cria uma figura (janela de gráfico) com 10 polegadas de largura e 6 de altura.
É o "container" que vai receber os dois subplots.

plt.subplot(2, 1, 1)

Divide a figura em 2 linhas, 1 coluna e seleciona o 1º painel (parte de cima).

plt.plot(dados[tempo_col], dados['Valor1'], label='Original', alpha=0.4)

Plota o sinal original do canal 1.

dados[tempo_col] = eixo X (tempo)

'Valor1' = eixo Y

label='Original' = legenda

alpha=0.4 = deixa a linha 40% transparente

```
plt.plot(dados[tempo_col], dados['Valor1_filtrado'], label=f'Filtrado {janela}', color='blue')
```

Plota o mesmo sinal após filtro de média móvel.

Valor1_filtrado = coluna criada anteriormente

color='blue' = cor azul fixa

label=f'Filtrado {janela}' = legenda mostra a janela usada (5,10,20...)

```
plt.title('Sinal 1')
```

```
plt.ylabel('Amplitude')
```

```
plt.ylim(0, 4000)
```

```
plt.xlim(0, 3000)
```

```
plt.legend()
```

```
ylim(0, 4000)
```

Define limite do eixo Y: amplitude vai de 0 a 4000

Isso cria uma escala fixa nos gráficos.

```
xlim(0, 3000)
```

Define limite do eixo X: tempo vai de 0 a 3000 ms

```
legend()
```

Ativa a legenda com:

Original

Filtrado (janela)

```
plt.subplot(2, 1, 2)
```

Agora o painel inferior é ativado (parte de baixo).

```
plt.plot(dados[tempo_col], dados['Valor2'], label='Original', alpha=0.4)
```

Mesmo que o sinal 1, porém para o segundo canal.

```
plt.plot(dados[tempo_col], dados['Valor2_filtrado'], label=f'Filtrado {janela}', color='green')
```

Agora o filtrado aparece em verde.

```
plt.title('Sinal 2')
```

```
plt.xlabel('Tempo (ms)')
```

```
plt.ylabel('Amplitude')
```

```
plt.ylim(0, 4000)
```

```
plt.xlim(0, 3000)
```

```
plt.legend()
```

Título do gráfico

Rótulos dos eixos

Escala fixa (mesma escala do primeiro gráfico)

Datasets:

Após a aplicação dos filtros de média móvel, os datasets foram suavizados e ajustados conforme

as diferentes janelas utilizadas, resultando em sinais filtrados que melhor se adequam às características do movimento analisado.

Dataset extensão_1 filtrado por média móvel 5:

A	B	C			
Tempo (ms)	Valor1_filtrado	Valor2_filtrado	701	0	30
3	0	0	702	0	17.6
5	0	0	703	0	17.6
7	0	0	704	0	17.6
8	0	0	705	0	4.8
9	0	0	706	0	0
10	0	0	707	0	0
11	0	0	708	0	0
12	0	0	709	0	2.6
13	0	0	710	0	2.6
14	0	0	711	0	2.6
15	0	0	712	0	35.4
16	0	0	713	0	42.4
17	0	0	714	0	39.8
18	0	0	715	0	61.8
19	0	0	716	0	61.8
20	0	0			

Dataset extensão_1 filtrado por média móvel 10:

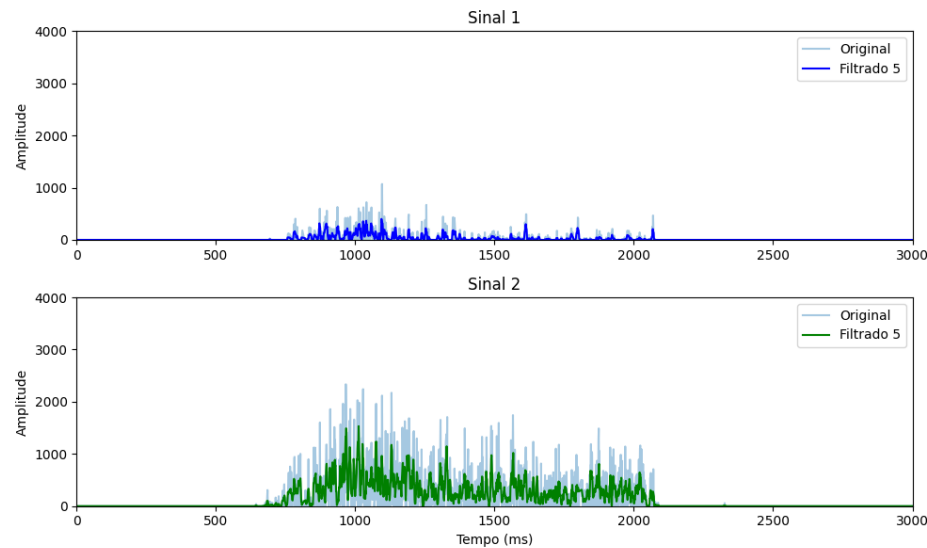
Tempo (ms)	Valor1_filtrado	Valor2_filtrado	701	0	19.3
3	0	0	702	0	19.3
5	0	0	703	0	15
7	0	0	704	0	15
8	0	0	705	0	8.8
9	0	0	706	0	8.8
10	0	0	707	0	10.1
11	0	0	708	0	3.7
12	0	0	709	0	1.3
13	0	0	710	0	17.7
14	0	0	711	0	21.2
15	0	0	712	0	21.2
16	0	0	713	0	32.2
17	0	0	714	0	32.2
18	0	0	715	0	32.2
19	0	0	716	0	32.2
			717	0	27.6

Dataset extensão_1 filtrado por média móvel 20:

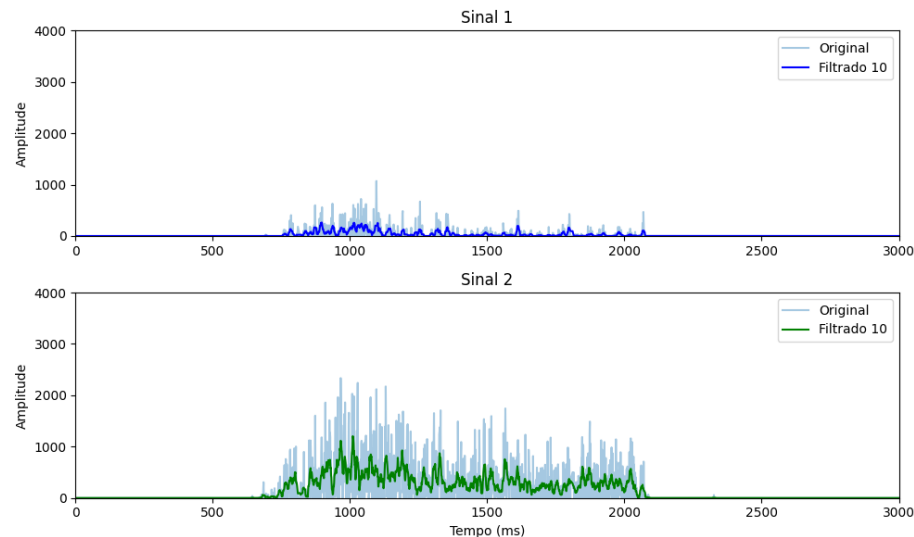
Tempo (ms)	Valor1_filtrado	Valor2_filtrado			
			701	2.2	14
3	0	0	702	2.2	14.65
5	0	0	703	2.2	14.65
7	0	0	704	0.8	14.65
8	0	0	705	0	18.5
9	0	0	706	0	20.25
10	0	0	707	0	20.25
11	0	0	708	0	23.6
12	0	0	709	0	23.6
13	0	0	710	0	20.5
14	0	0	711	0	20.5
15	0	0	712	0	21.35
16	0	0	713	0	18.15
17	0	0	714	0	16.95
18	0	0	715	0	17
19	0	0	716	0	26.5
			717	0	26.5

A seguir, também é demonstrado como o gráfico foi modificado após a filtragem, apresentando-se mais limpo e suavizado. Essa melhoria visual facilita significativamente a análise dos movimentos, permitindo observar com clareza as variações do sinal em diferentes níveis de janelas de filtragem.

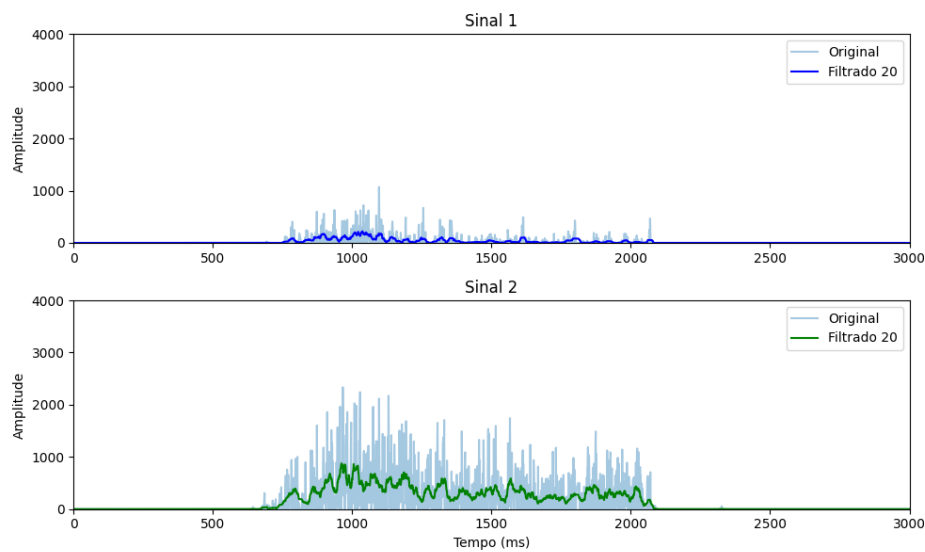
Gráficos extensão_1 média móvel de 5:



Gráficos extensão_1 média móvel de 10:



Gráficos extensão_1 média móvel de 20:

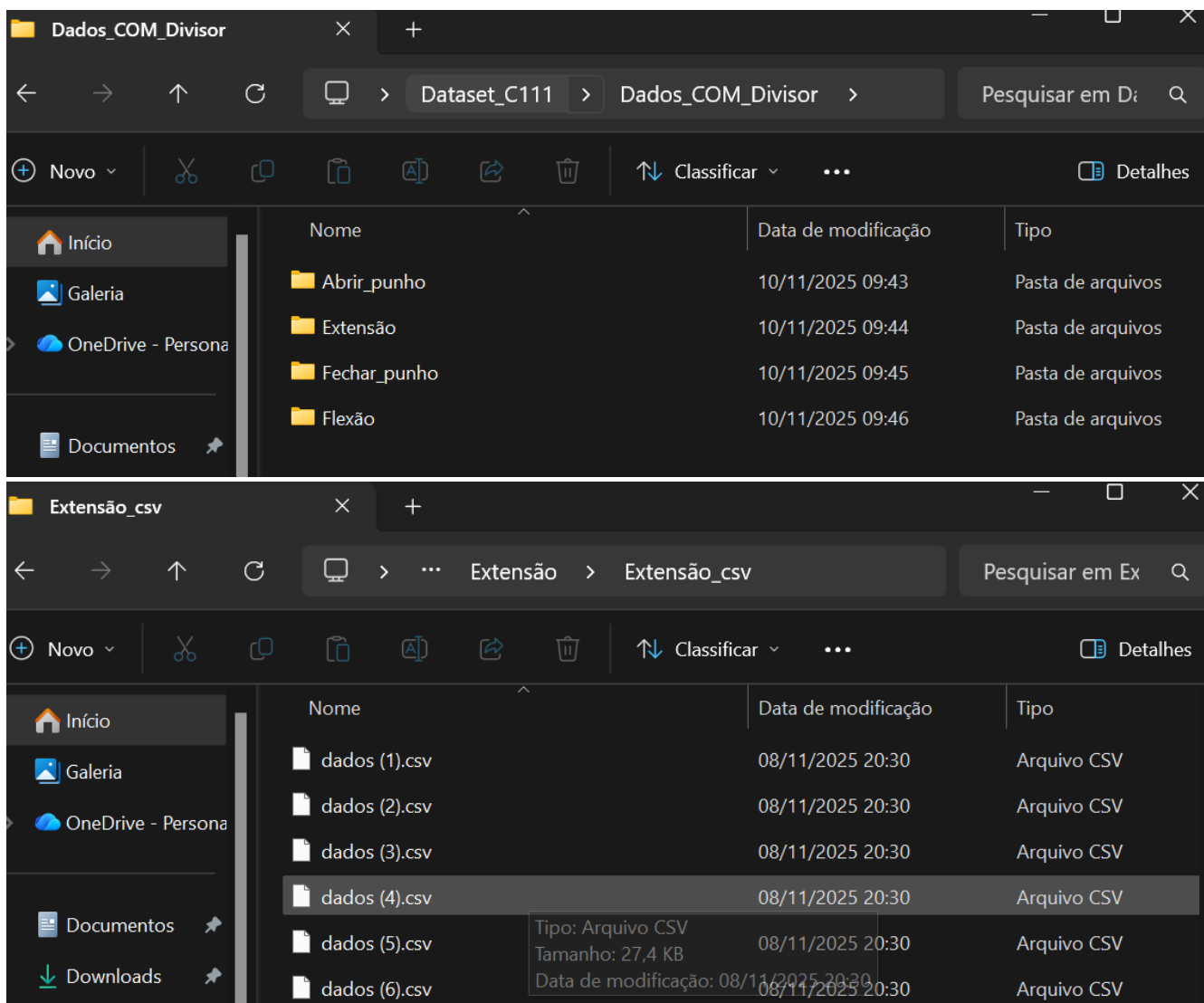


Organização do nosso banco de dados com os dataset :

Os dados foram organizados em pastas separadas de acordo com o movimento executado.

Para cada movimento (abrir punho, fechar punho, flexão e extensão), foram coletados aproximadamente “15” arquivos CSV contendo a leitura dos dois canais EMG.

Dentro de cada pasta, os arquivos têm nomes padronizados (ex.: dados_01.csv, dados_02.csv, etc.), o que facilita o processamento automático.



3. Análise estatística e visualização

Para aprimorar a interpretação visual dos dados gerados e avaliar a eficácia dos filtros aplicados na identificação dos movimentos musculares, adicionamos alguns parâmetros adicionais aos novos arquivos .csv resultantes da aplicação da média móvel.

Optamos por utilizar janelas de tamanho **10**, pois apresentaram a melhor suavização dos sinais sem perda significativa de informações relevantes. Além disso, calculamos a **média aritmética** dos sinais para cada movimento, permitindo a geração de valores representativos que facilitaram a construção do gráfico de **scatter plot**, auxiliando na observação da distribuição e separação dos dados.

Também foi calculada a **média RMS (Root Mean Square)**, com o objetivo de avaliar a variação da amplitude dos sinais e evidenciar diferenças entre os movimentos, contribuindo para uma análise mais precisa e comparativa.

Media Aritmética:

A média aritmética é o valor que você obtém **somando todos os elementos** e depois **dividindo pelo número de elementos**.

$$\bar{x} = \frac{x_1 + x_2 + x_3 + \dots + x_n}{n}$$

Em sinais (como EMGs):

pegamos um conjunto de pontos próximos no tempo

calculamos a média

usamos esse valor para substituir o ponto central

Isso suaviza o sinal eliminando variações rápidas (ruído).

Media RMS:

A **média RMS** (Root Mean Square), também chamada de **valor quadrático médio** ou **valor eficaz**, é uma forma de medir o *tamanho efetivo* de um sinal que varia no tempo.

Ela é muito usada em sinais elétricos, eletrônicos, vibração, áudio, EMG, etc., porque fornece uma noção real da *energia média* do sinal.

$$RMS = \sqrt{\frac{1}{n} \sum_{i=1}^n x_i^2}$$

A média tradicional (aritmética) deixa valores positivos e negativos se cancelarem.

No EMG, por exemplo, se você fizer a média aritmética de um sinal oscilante, pode dar **zero**, e não significa que não há atividade muscular — apenas que o sinal oscila ao redor de zero.

A **média RMS não cancela** positivo com negativo, pois antes de calcular a média os valores são **quadrados**.

Em sinais EMG, o RMS é muito usado porque:

suaviza o sinal

reduz ruído

mostra intensidade muscular

permite detectar picos de contração

facilita comparar movimentos diferentes

Comparação dos sinais após serem filtrados pela média móvel de janela de 10: dados de coleta 1 :

Gráfico sinal extensor_1 janela de 10:

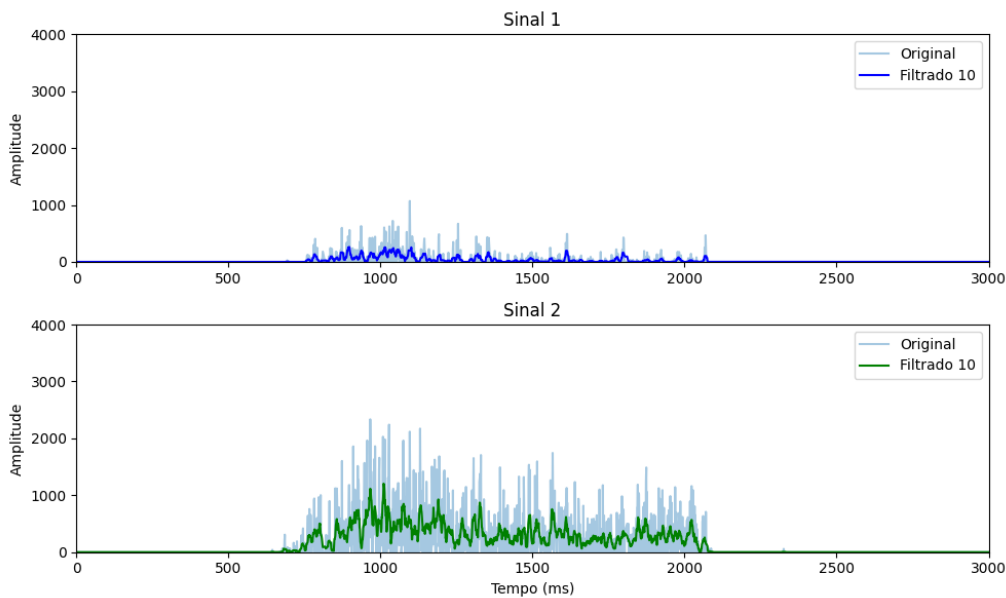


Gráfico sinal flexor_1 janela de 10:

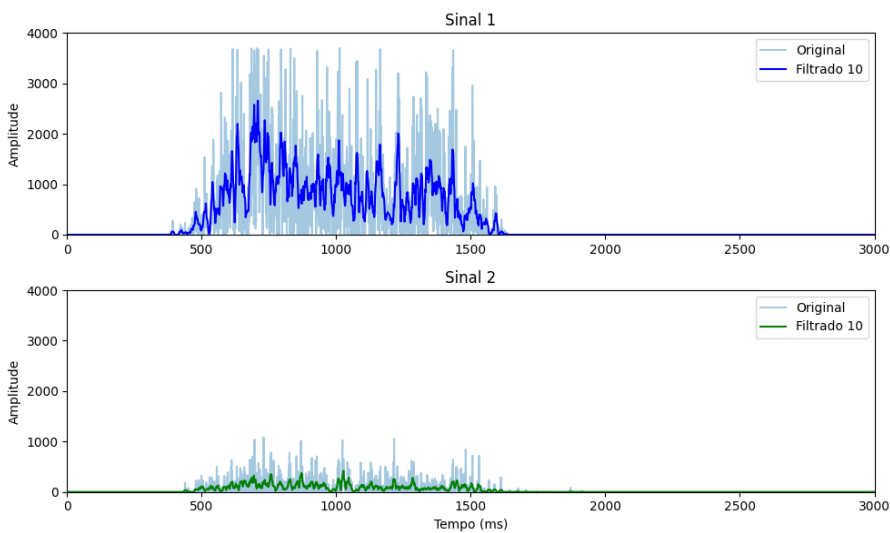


Gráfico sinal abrir punho_1 janela de 10:

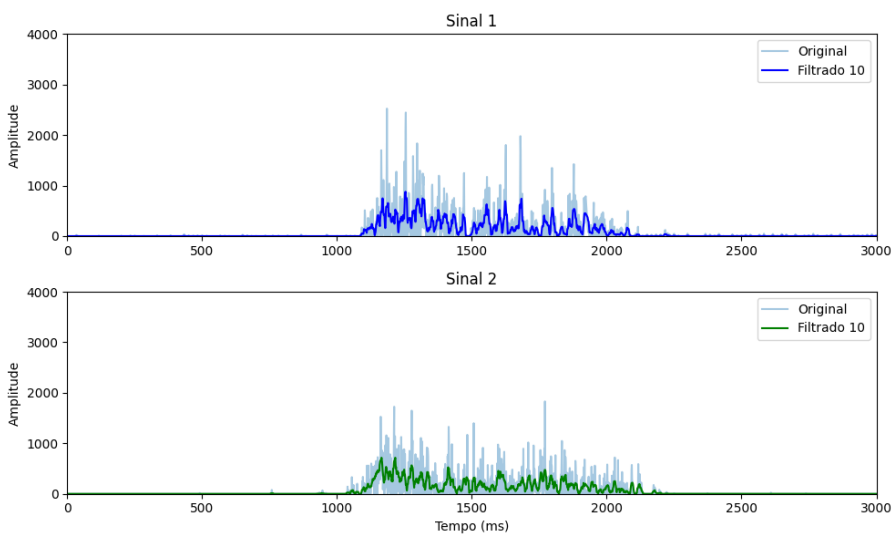
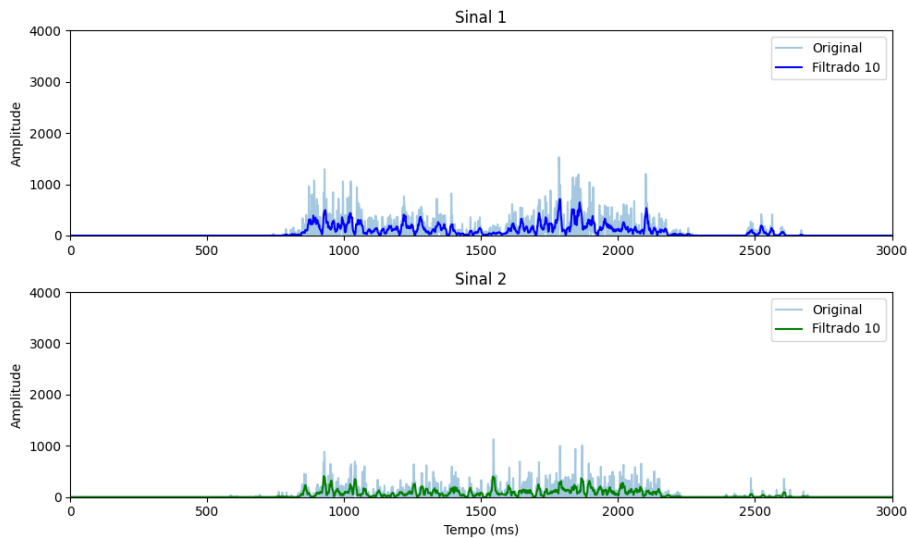


Gráfico sinal fechar punho_1 janela de 10:



Média Aritmética e média RMS Códigos:

Os códigos utilizados para o cálculo da média RMS e da média aritmética seguiram uma metodologia baseada no processamento direto de cada amostra filtrada após a aplicação da média móvel. A abordagem consistiu em realizar os cálculos de forma individual para cada conjunto de dados filtrados, garantindo precisão e coerência na análise estatística.

A proposta é extrair os valores resultantes das médias RMS e aritméticas de todos os datasets e organizá-los em uma tabela estruturada. Essa tabela servirá como base para a construção de um **scatter plot** comparativo, no qual serão representados os quatro movimentos analisados. Dessa forma, torna-se possível visualizar, em um único gráfico, a distribuição e a região característica de cada movimento, facilitando a identificação e diferenciação entre eles.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

# ---- 1. Entrada manual do intervalo ----
inicio_tempo = float(input("Digite o tempo de início do sinal: "))
fim_tempo = float(input("Digite o tempo de fim do sinal: "))
```

Neste trecho do código, realizamos uma etapa de captura manual na qual, de forma visual, identificamos e atribuiremos para cada segmento de dados os valores correspondentes aos pontos de entrada e saída na escala temporal (eixo X). Essa marcação permite delimitar com precisão os intervalos relevantes para análise, garantindo que cada movimento seja corretamente segmentado e interpretado.

```

# ---- 3. Considerar colunas do CSV: tempo, valor1, valor2 ----
tempo = df.iloc[:, 0].values
valor1 = df.iloc[:, 1].values
valor2 = df.iloc[:, 2].values

# ---- 4. Encontrar índices correspondentes ao intervalo manual ----
inicio_idx = np.argmin(np.abs(tempo - inicio_tempo))
fim_idx = np.argmin(np.abs(tempo - fim_tempo))

# ---- 5. Função para calcular RMS e energia média ----
def calcular_metricas(sinal, inicio_idx, fim_idx):
    trecho = sinal[inicio_idx:fim_idx]
    rms = np.sqrt(np.mean(trecho**2)) # RMS correto
    energia_media = np.mean(trecho)
    return rms, energia_media

rms1, energia1 = calcular_metricas(valor1, inicio_idx, fim_idx)
rms2, energia2 = calcular_metricas(valor2, inicio_idx, fim_idx)

```

O código começa selecionando as três colunas principais do arquivo CSV: tempo, valor1 e valor2. Isso significa que ele está extraindo os dados de tempo e os dois sinais (geralmente os dois canais de EMG) para trabalhar separadamente. Depois disso, o código identifica os índices que correspondem ao intervalo selecionado manualmente pelo usuário, comparando os valores do vetor de tempo com os limites definidos (início e fim), retornando o ponto mais próximo no eixo temporal.

Em seguida, é definida uma função chamada **calcular_metricas**, que recebe um sinal e os índices de início e fim. Dentro dessa função, o trecho do sinal dentro do intervalo selecionado é extraído. A partir desse trecho, o código calcula duas métricas:

- **RMS**, que é obtido elevando os valores ao quadrado, tirando a média desses valores e, por fim, extraindo a raiz quadrada — representando a intensidade do sinal.
- **Energia média**, que é simplesmente a média aritmética do trecho, indicando o valor médio do sinal naquele intervalo.

Por fim, a função é aplicada aos dois sinais (valor1 e valor2), gerando o RMS e a energia média para cada canal. Isso permite comparar a força do sinal e a atividade muscular dentro do trecho específico marcado pelo usuário.

```

# ---- 6. Função para plotar e salvar sinal ----
def plotar_sinal_manual(tempo, sinal, inicio_idx, fim_idx, rms, energia, titulo, cor='b', filename="grafico.png"):
    plt.figure(figsize=(12,6))
    plt.plot(tempo, sinal, label=titulo, color=cor)

    # Linhas verticais de início e fim
    plt.axvline(tempo[inicio_idx], color='r', linestyle='--', label='Início')
    plt.axvline(tempo[fim_idx], color='g', linestyle='--', label='Fim')

    # Anotar valores de início e fim no gráfico
    plt.annotate(f"{tempo[inicio_idx]:.2f}", xy=(tempo[inicio_idx], sinal[inicio_idx]),
                xytext=(tempo[inicio_idx], max(sinal)*0.9),
                arrowprops=dict(arrowstyle="->", color='red'), color='red')
    plt.annotate(f"{tempo[fim_idx]:.2f}", xy=(tempo[fim_idx], sinal[fim_idx]),
                xytext=(tempo[fim_idx], max(sinal)*0.9),
                arrowprops=dict(arrowstyle="->", color='green'), color='green')

    # Tabela no canto superior esquerdo
    tabela_valores = [{"RMS", f"{rms:.4f}"},
                     ["Media Aritmetica", f"{energia:.4f}"]]
    plt.table(cellText=tabela_valores, loc='upper left', cellLoc='center', colWidths=[0.2,0.2])

    # Fixar o eixo Y de 0 a 4000
    plt.ylim(0, 4000)

    plt.title(titulo)
    plt.xlabel("Tempo")
    plt.ylabel("Amplitude")
    plt.legend()
    plt.grid(True)

    # Salvar gráfico
    plt.savefig(filename, dpi=300, bbox_inches='tight')
    plt.close()

# ---- 7. Plotar e salvar os dois sinais ----
plotar_sinal_manual(tempo, valor1, inicio_idx, fim_idx, rms1, energia1, "Sinal 1", cor='b', filename="sinal1.png")
plotar_sinal_manual(tempo, valor2, inicio_idx, fim_idx, rms2, energia2, "Sinal 2", cor='orange', filename="sinal2.png")

```

O código define uma função responsável por gerar um gráfico personalizado de um sinal e, ao mesmo tempo, salvar a imagem automaticamente. A função recebe como entrada os vetores de tempo e sinal, os índices que delimitam o intervalo selecionado, os valores de RMS e energia média calculados anteriormente, o título do gráfico e a cor da linha. Também é possível especificar o nome do arquivo a ser salvo.

Dentro da função, o primeiro passo é criar uma figura com um tamanho definido. Em seguida, o sinal é plotado completo ao longo do tempo. Para marcar visualmente o trecho analisado, duas linhas verticais são desenhadas: uma representando o início do intervalo e outra representando o fim. Essas linhas auxiliam na identificação visual do ponto do sinal considerado para cálculo das métricas.

Além das linhas, o código adiciona anotações diretamente no gráfico, indicando numericamente os valores exatos do tempo nos pontos de início e fim. Essas anotações possuem setas que apontam para o local correspondente no gráfico, facilitando a leitura e interpretação.

No canto superior esquerdo do gráfico, o código insere uma pequena tabela contendo dois valores importantes: o RMS (que representa a força ou intensidade média do sinal) e a média aritmética (chamada de energia média). Essa tabela permite visualizar rapidamente as métricas associadas ao trecho selecionado.

Também é aplicado um limite fixo para o eixo Y, definido entre 0 e 4000, garantindo uma escala padronizada para a visualização e facilitando a comparação entre diferentes gráficos. Títulos, legendas, rótulos de eixos e grade são adicionados para tornar o gráfico mais completo e organizado.

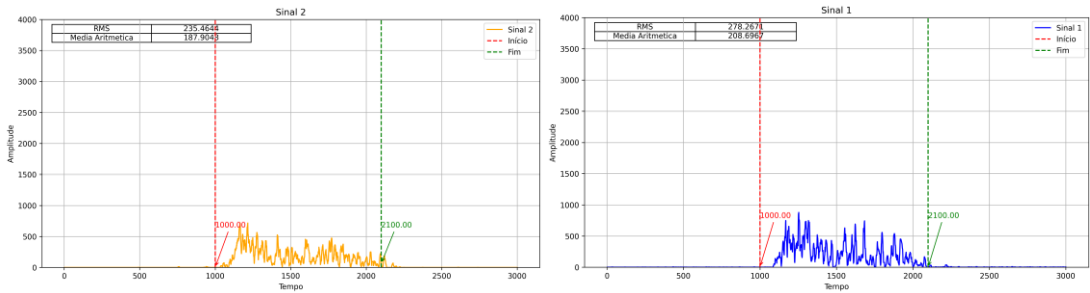
No final do processo, em vez de apenas exibir o gráfico, a função salva a figura diretamente em um arquivo de imagem usando o nome especificado. Depois de salvar, a figura é fechada para evitar sobreposição de gráficos durante execuções repetidas.

Por fim, o código chama a função duas vezes: uma para gerar o gráfico do primeiro sinal e outra para o segundo sinal, salvando cada imagem separadamente com os nomes “sinal1.png” e “sinal2.png”. Dessa forma, ambos os gráficos são processados, anotados, salvos e organizados de maneira automática.

Gráficos após passarem pelo filtro de média aritmética e média rms :
Dados filtrados_1:

Grafico Abrir-punho_1 média aritmética e média RMS :

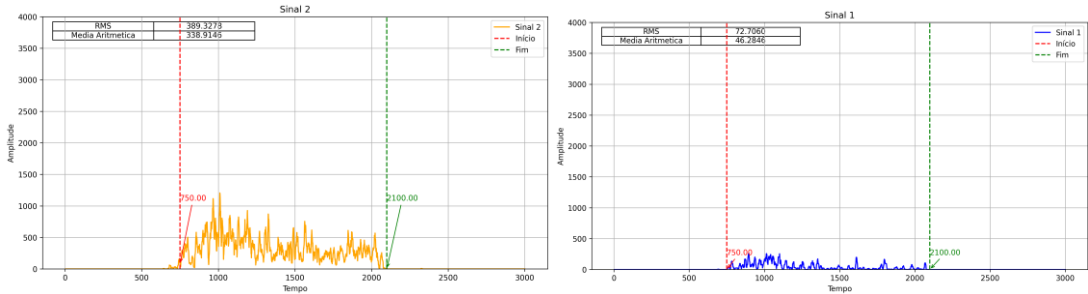
```
*** Digite o tempo de início do sinal: 1000
Digite o tempo de fim do sinal: 2100
Faça o upload do(s) arquivo(s) CSV para análise.
Escolher arquivos dados (1)_filtrado_10.csv
dados (1)_filtrado_10.csv(text/csv) - 40871 bytes, last modified: 10/11/2025 - 100% done
Saving dados (1)_filtrado_10.csv to dados (1)_filtrado_10.csv
✅ Arquivos gerados: sinal1.png, sinal2.png e media.csv
```



	A	B	C	D
1	Canal	RMS	EnergiaMedia	
2	Valor1	278.2671	208.6967	
3	Valor2	235.4644	187.9048	
4				
5				

Grafico Extensão_1 média aritmética e média RMS :

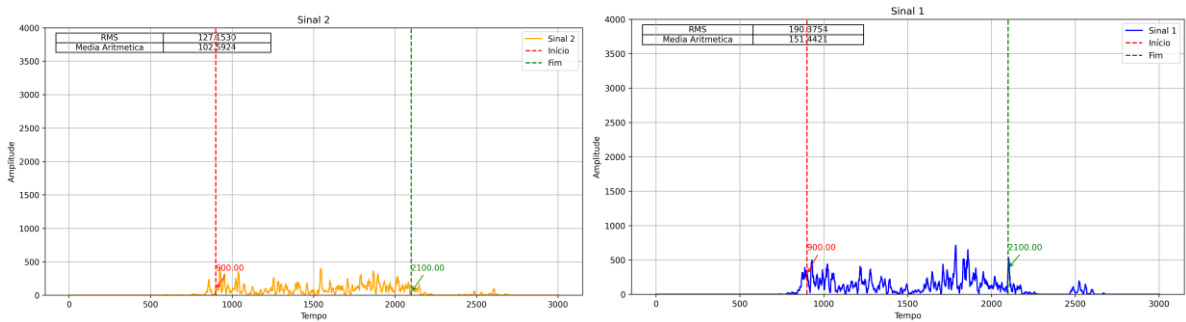
```
... Digite o tempo de início do sinal: 750
Digite o tempo de fim do sinal: 2100
Faça o upload do(s) arquivo(s) CSV para análise.
Escolher arquivos dados (1)_filtrado_10.csv
dados (1)_filtrado_10.csv(text/csv) - 41116 bytes, last modified: 10/11/2025 - 100% done
Saving dados (1)_filtrado_10.csv to dados (1)_filtrado_10 (1).csv
✓ Arquivos gerados: sinal1.png, sinal2.png e media.csv
```



	A	B	C	D
1	Canal	RMS	Energia	Media
2	Valor1	72.70603	46.28464	
3	Valor2	389.3278	338.9146	
4				

Grafico fechar-punho_1 média aritmética e média RMS :

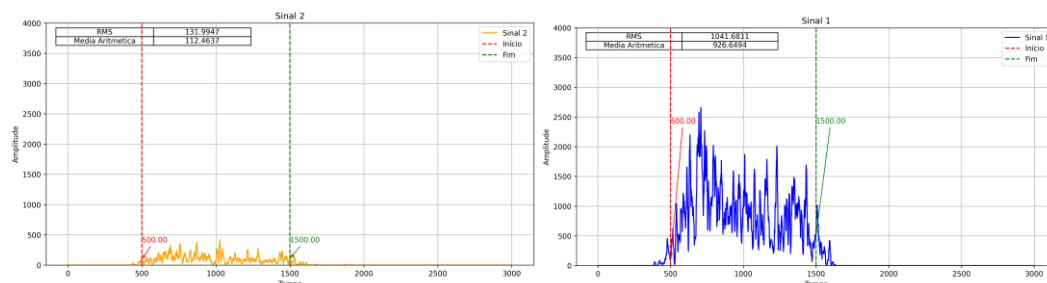
```
... Digite o tempo de início do sinal: 900
Digite o tempo de fim do sinal: 2100
Faça o upload do(s) arquivo(s) CSV para análise.
Escolher arquivos dados (1)_filtrado_10.csv
dados (1)_filtrado_10.csv(text/csv) - 41509 bytes, last modified: 10/11/2025 - 100% done
Saving dados (1)_filtrado_10.csv to dados (1)_filtrado_10 (1).csv
✓ Arquivos gerados: sinal1.png, sinal2.png e media.csv
```



	A	B	C	
1	Canal	RMS	Energia	Media
2	Valor1	190.3754	151.4421	
3	Valor2	127.153	102.5924	
4				

Grafico Flexão_1 média aritmética e média RMS :

```
... Digite o tempo de início do sinal: 500
Digite o tempo de fim do sinal: 1500
Faça o upload do(s) arquivo(s) CSV para análise.
Escolher arquivos dados (1)_filtrado_10.csv
dados (1)_filtrado_10.csv(text/csv) - 41563 bytes, last modified: 10/11/2025 - 100% done
Saving dados (1)_filtrado_10.csv to dados (1)_filtrado_10 (1).csv
✓ Arquivos gerados: sinal1.png, sinal2.png e media.csv
```



	A	B	C
1	Canal	RMS	EnergiaMedia
2	Valor1	1041.681	926.6494
3	Valor2	131.9947	112.4637
4			

Essas análises foram aplicadas a todos os dados filtrados pela média móvel, abrangendo os 15 registros de flexão, os 15 registros de extensão, e os 15 registros correspondentes aos movimentos de abrir e fechar o punho.

Planilha de média aritmética:

	EXTENSÃO		FLEXÃO		ABRIR_PUNHO		FECHAR_PUNHO	
	sinal_1	sinal_2	sinal_1	sinal_2	sinal_1	sinal_2	sinal_1	sinal_2
1	22.87297921	284.2204208	723.9761076	92.98911392	208.6967	187.9048	151.4421	102.5924
2	104.1592437	235.4444282	1163.530788	88.3911032	318.4995	228.9857	159.3713	48.6704
3	69.967891	196.5660309	986.0306142	100.8193467	184.7874	198.0113	214.1227	100.2474
4	23.3318232	148.6276596	689.4151351	48.63247041	212.7954	141.2559	311.0192	139.3443
5	138.3706587	157.7293103	731.1052542	34.13268041	173.2456	181.7595	250.5787	144.4322
6	47.37713718	228.0302916	592.7716073	73.34846626	150.5821	146.9589	511.6526	151.7258
7	120.9309211	175.8698783	621.0121764	62.82120658	177.4304	147.0762	217.5288	85.1465
8	56.9127572	280.4711165	823.8921279	38.61559633	216.1849	195.0268	224.4061	67.4256
9	62.96486486	162.8889423	591.6548694	67.171875	329.1861	170.4632	211.9732	85.1853
10	39.01791198	307.6779221	797.5358944	66.36864686	202.8154	130.9741	202.4584	82.838
11	65.81597633	281.5848769	670.2451081	70.73956186	271.9661	126.6292	259.4665	128.3585
12	92.69324324	136.5973618	628.4773087	52.85985222	277.7525	154.7902	225.2164	108.0218
13	42.29876543	184.9956938	724.8120527	82.34411765	407.8583	189.2239	399.0036	126.7782
14	123.8214286	167.6223714	661.51	74.50693333	146.9489	154.5998	351.3202	113.4271
15	124.2944444	127.5179582	676.062069	60.98459959	209.3746	148.2793	299.5881	90.7736

Com esses valores conseguimos plotar o gráfico de dispersão para identificar a diferença entre os movimentos.

O que é um gráfico de scatter plot (gráfico de dispersão):

O **scatter plot** é um tipo de gráfico formado por **pontos individuais**, cada ponto representando um par de valores entre duas variáveis.

Ele é usado para analisar:

relação entre variáveis
correlação
agrupamentos (clusters)
padrões de comportamento
presença de outliers

Cada ponto representa uma variável x e y

X → valor de uma variável

Y → valor da outra variável

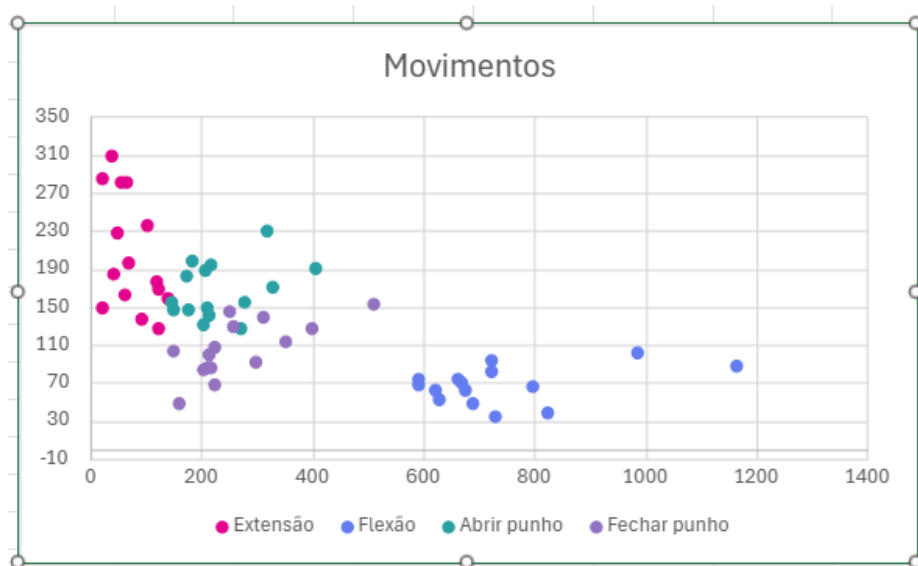
O gráfico de dispersão (scatter plot) foi utilizado para analisar a relação entre duas variáveis do sinal EMG.

Cada ponto representa uma amostra simultânea dos dois canais de medição.

Esse tipo de gráfico permite visualizar se existe correlação entre os sensores, identificar padrões de distribuição, regiões de maior densidade e eventuais outliers.

Sua utilização facilita a compreensão do comportamento muscular e das diferenças entre movimentos analisados.

Gráfico de dispersão:



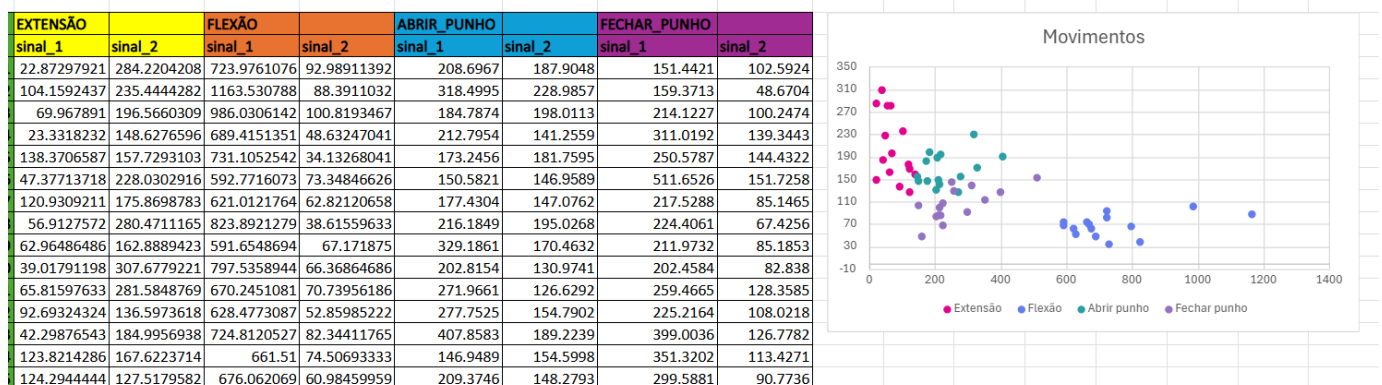
Com o gráfico de dispersão, foi possível visualizar de forma eficiente as diferenças entre os sinais elétricos associados aos movimentos de **abrir punho**, **fechar punho**, **extensão** e **flexão**. A representação gráfica evidenciou que, utilizando dois sensores para a captação dos sinais, torna-se viável distinguir cada movimento a partir das variações presentes no dataset.

Esse resultado foi alcançado após a aplicação de etapas de processamento que incluíram:

- uso de um **divisor de tensão** para reduzir saturações e minimizar ruído,
- aplicação do **filtro de média móvel** com janelas de 5, 10 e 20 amostras,

- cálculo da **média aritmética** e da **média RMS**, permitindo estruturar uma tabela consolidada dos parâmetros extraídos.

Com esses dados organizados, foi então possível gerar um gráfico de dispersão capaz de separar visualmente os movimentos, demonstrando a eficiência do processo de filtragem e análise aplicado ao dataset.



4 – Análise Crítica

Insight 1 – Diferença de amplitude entre os movimentos

Os gráficos filtrados pela média móvel com janela de 10 mostraram que os movimentos de **flexão e extensão** apresentaram amplitudes médias maiores em comparação aos movimentos de **abrir e fechar o punho**.

Esse comportamento pode ser explicado pelo **maior esforço muscular envolvido na flexão e extensão**, que gera uma ativação elétrica mais intensa.

Além disso, observou-se que durante a **extensão**, o **sensor 1** apresentou valores mais altos, enquanto durante a **flexão**, o **sensor 2** teve maior amplitude — o que indica que cada sensor responde de forma distinta conforme o músculo ativado.

Essa diferença permite identificar tipo de movimento, mas também a **posição relativa dos sensores**, auxiliando no reconhecimento.

Insight 2 – Redução de ruído com a filtragem por média móvel

Antes da aplicação dos filtros, os sinais apresentavam **oscilações rápidas e picos isolados** que dificultavam a interpretação do comportamento muscular.

Com a utilização do **filtro de média móvel** (principalmente com **janela de 10**), houve uma **redução significativa do ruído elétrico** sem comprometer as informações fisiológicas relevantes.

O sinal tornou-se mais contínuo e coerente com o comportamento real dos músculos, mostrando que a filtragem foi eficiente para eliminar interferências e melhorar a clareza das leituras.

Insight 3 – Estabilidade e repetibilidade do sinal

Ao analisar coletas repetidas de um mesmo movimento, observou-se que o padrão do sinal filtrado mantém comportamento semelhante entre diferentes execuções, indicando estabilidade e reprodutibilidade do sistema de aquisição.

Pequenas variações de amplitude entre as coletas foram identificadas, o que pode estar relacionado à variação da pressão dos eletrodos ou ajuste de posicionamento no braço.

Mesmo assim, o formato e a tendência geral dos sinais se mantiveram consistentes, confirmando a confiabilidade do sistema de captação e processamento.

Síntese da análise

Os resultados demonstram que a metodologia de **filtragem, cálculo de médias e análise gráfica** foi eficaz para extrair informações relevantes dos sinais EMG.

A combinação das técnicas de **média móvel, RMS e gráficos de dispersão** possibilitou **reduzir ruído, identificar padrões de ativação muscular e diferenciar movimentos**, validando o potencial do sistema como base para aplicações futuras em controle de próteses inteligentes.

Observação de possíveis limitações da análise

Apesar dos resultados positivos obtidos, algumas limitações foram identificadas durante o processo de coleta e análise dos sinais mioelétricos.

1. Precisão do hardware de aquisição

O microcontrolador **ESP32**, embora versátil e de baixo custo, possui **limitações no conversor analógico-digital (ADC)**, tanto em resolução quanto na taxa de amostragem. Isso pode gerar **pequenas distorções** ou **perda de detalhes** em sinais de alta frequência, afetando a fidelidade da leitura.

2. Ruídos e interferências externas

Mesmo após a aplicação dos filtros digitais, ainda é possível observar **resquícios de ruído elétrico**, possivelmente provenientes da alimentação, do ambiente eletromagnético ou de **imperfeições no aterramento**. Essas interferências reduzem a pureza do sinal captado.

3. Variações no posicionamento dos sensores

Pequenas diferenças na **posição e pressão dos eletrodos** entre as coletas podem alterar a amplitude e a resposta elétrica do músculo. Isso afeta a **comparabilidade direta** entre diferentes execuções do mesmo movimento.

4. Quantidade limitada de coletas por movimento

Embora o número de arquivos por movimento (cerca de 15) seja suficiente para análise inicial, um **número maior de repetições** poderia melhorar a **robustez estatística** e a **generalização dos resultados**, principalmente para uso futuro em redes neurais.

5. Ausência de sincronização temporal perfeita

A **irregularidade na amostragem temporal** observada nos datasets impede a aplicação de algumas técnicas avançadas de análise no domínio da frequência, como transformadas de Fourier precisas.

• 6. Fatores fisiológicos humanos

Variações naturais entre contrações musculares, **fadiga, tensão do braço e posição do membro** também influenciam a amplitude e o padrão do sinal EMG, dificultando a padronização completa dos resultados.

5. Processo de colaboração com a IA

Registro das interações com IA incluindo:

Prompts utilizados:

Respostas recebidas:

O que foi aproveitado, descartado ou modificado:

Objetivo: Criar a primeira versão do código responsável por ler sinais EMG (ou outros dados em CSV), aplicar um filtro de média móvel simples e salvar o resultado, servindo como ponto de partida para evoluções futuras.

"Quero iniciar o desenvolvimento de um código em Python para processar sinais EMG salvos em CSV. O código deve ler o arquivo CSV, verificar automaticamente se a coluna de tempo se chama 'Tempo (ms)' ou 'Tempo', e aplicar um filtro de média móvel em duas colunas ('Valor1' e 'Valor2'). Depois disso, ele deve salvar o resultado em um novo arquivo CSV.

Crie uma função chamada `aplicar_filtro_e_salvar(nome_arquivo, janela)` que:

- "Leia o CSV;"
- "Verifique o nome da coluna de tempo (aceitando 'Tempo (ms)' ou 'Tempo');"
- "Aplique o filtro de média móvel com uma janela configurável (ex: 5, 10 ou 20);"
- "Salve o novo CSV com o sufixo `_filtrado`."

Deixe o código bem comentado e fácil de entender, pois ele será o ponto inicial do projeto."

python

 Copiar código

```
import pandas as pd

# ---- Função para filtrar e salvar ----
def aplicar_filtro_e_salvar(nome_arquivo, janela):
    # Carrega CSV
    dados = pd.read_csv(nome_arquivo)

    # Checa nome das colunas de tempo
    if "Tempo (ms)" in dados.columns:
        tempo_col = "Tempo (ms)"
    elif "Tempo" in dados.columns:
        tempo_col = "Tempo"
    else:
        raise KeyError("Coluna de tempo não encontrada (esperado: 'Tempo (ms)' ou 'Tempo')")

    # Aplica filtro de média móvel
    dados['Valor1_filtrado'] = dados['Valor1'].rolling(window=janela, center=True, min_periods=1).
    dados['Valor2_filtrado'] = dados['Valor2'].rolling(window=janela, center=True, min_periods=1).

    # Salva novo CSV
    nome_saida = nome_arquivo.replace('.csv', f'_filtrado_{janela}.csv')
    dados.to_csv(nome_saida, index=False)

    print(f"✅ Arquivo filtrado salvo como: {nome_saida}")
```

O código base funciona, mas ainda não gera gráficos nem métricas de RMS e energia.

Essa primeira versão serve apenas para validar a filtragem e a leitura do CSV.

Decisão: **aceitar o código** como versão 1 e planejar para a próxima etapa:

Incluir cálculo de RMS e energia média;

Adicionar geração de gráficos (com intervalo definido manualmente).

Objetivo: evoluir o código da primeira versão (que apenas filtrava e salvava) para agora incluir **análise quantitativa** dos sinais — calculando **RMS e energia média** em um intervalo definido manualmente pelo usuário.

O foco é permitir análise controlada dos dados após filtragem.

"Agora quero evoluir o código anterior.

Adicione a funcionalidade para que o usuário digite o tempo de início e fim do sinal (valores em segundos ou milissegundos, conforme a coluna do CSV).

O código deve:

- "Solicitar `inicio_tempo` e `fim_tempo` via `input()`;"
- "Identificar os índices correspondentes no eixo de tempo;"
- "Calcular RMS e Energia média para as colunas 'Valor1' e 'Valor2' nesse intervalo;"
- "Mostrar os valores no console e salvar tudo em um CSV chamado `metricas.csv`;"
- "Continuar utilizando a lógica de filtragem anterior, mas sem gerar gráficos ainda."

Inclua comentários claros, mantendo o estilo organizado e fácil de entender."

```

python

import pandas as pd
import numpy as np

# ---- 1. Solicitar intervalo manual ----
inicio_tempo = float(input("Digite o tempo de início do sinal: "))
fim_tempo = float(input("Digite o tempo de fim do sinal: "))

# ---- 2. Função para aplicar filtro e calcular métricas ----
def aplicar_filtro_e_metricas(nome_arquivo, janela):
    dados = pd.read_csv(nome_arquivo)

    # Detecta nome da coluna de tempo
    if "tempo (ms)" in dados.columns:
        tempo_col = "tempo (ms)"
    elif "tempo" in dados.columns:
        tempo_col = "tempo"
    else:
        raise KeyError("coluna de tempo não encontrada.")

    tempo = dados[tempo_col].values
    valor1 = dados["valor1"].rolling(window=janela, center=True, min_periods=1).mean().values
    valor2 = dados["valor2"].rolling(window=janela, center=True, min_periods=1).mean().values

    # ---- 3. Encontrar índices correspondentes ----
    inicio_idx = np.argmax(np.abs(tempo - inicio_tempo))
    fim_idx = np.argmax(np.abs(tempo - fim_tempo))

    # ---- 4. Função para calcular RMS e Energia Média ----
    def calcular_metricas(sinal, i1, i2):
        trecho = sinal[i1:i2]
        rms = np.sqrt(np.mean(trecho**2))
        energia_media = np.mean(trecho)
        return rms, energia_media

    rms1, energia1 = calcular_metricas(valor1, inicio_idx, fim_idx)
    rms2, energia2 = calcular_metricas(valor2, inicio_idx, fim_idx)

    # ---- 5. Salvar resultados ----
    df_metricas = pd.DataFrame({
        "Canal": ["valor1", "valor2"],
        "RMS": [rms1, rms2],
        "Energia Média": [energia1, energia2]
    })

    nome_saida = nome_arquivo.replace(".csv", f"_metricas_{janela}.csv")
    df_metricas.to_csv(nome_saida, index=False)

    print(f"📄 Métricas salvas em: {nome_saida}")
    print(df_metricas)

# ---- 6. Executar função ----
aplicar_filtro_e_metricas("sinal_exemplo.csv", janela=10)

```

A função agora calcula corretamente RMS e energia média no intervalo definido pelo usuário.

A identificação automática da coluna de tempo foi mantida, garantindo compatibilidade com CSVs diferentes.

O código continua modular e claro, mantendo a separação entre leitura, filtragem e cálculo.

Ajustes planejados para a próxima versão:

O código ainda **não gera gráficos** (somente salva as métricas).

É necessário permitir **upload direto pelo Colab** para evitar erro de caminho manual.

A próxima versão deve incluir **visualização dos sinais com linhas verticais** mostrando o intervalo analisado e anotações dos valores.

Aceitamos essa versão como **V2 Análise Quantitativa Básica**, pois implementa as métricas com sucesso e prepara a base para visualização.

Objetivo: evoluir o código da Versão 2 (que calcula RMS e energia média) para agora **plotar os sinais filtrados com marcações visuais do intervalo de análise**, além de salvar automaticamente os gráficos em formato `.png`.

"Agora quero que o código anterior gere gráficos para cada canal ('Valor1' e 'Valor2').

O gráfico deve conter:

- "O sinal ao longo do tempo;"
- "Duas linhas verticais marcando o início (`início_tempo`) e o fim (`fim_tempo`) do intervalo;"
- "Anotações mostrando os valores exatos de tempo onde começam e terminam as medições;"
- "Uma tabela no canto superior esquerdo contendo os valores de RMS e Energia Média;"
- "Eixo Y fixado de 0 a 4000 para manter comparação padronizada entre diferentes sinais;"
- "Gráficos salvos automaticamente em alta resolução (`.png`)."

O código deve continuar salvando o CSV com métricas e agora gerar dois gráficos:

`sinal1.png` e `sinal2.png`.

Mantenha o estilo claro e bem comentado, pois ele será usado em relatório técnico."

```
python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

# ---- 1. Entrada manual de intervalo ----
início_tempo = float(input("Digite o tempo de início do sinal: "))
fim_tempo = float(input("Digite o tempo de fim do sinal: "))

# ---- 2. Upload de arquivos CSV ----
print("Carregando arquivos CSV...")
arquivos = files.upload()
df = pd.read_csv(arquivos)

# ---- 3. Identificar colunas ----
tempo = df.iloc[:, 1].values
valor1 = df.iloc[:, 2].values
valor2 = df.iloc[:, 3].values

# ---- 4. Localizar índices correspondentes ao intervalo ----
início_idx = np.where(tempo == início_tempo)[0][0]
fim_idx = np.where(tempo == fim_tempo)[0][0]

# ---- 5. Calcular métricas ----
def calcular_metricas(sinal, li, li2):
    trecho = sinal[li:li2]
    rms = np.sqrt(np.mean(trecho**2))
    energia = np.mean(trecho)
    return rms, energia

rms1, energia1 = calcular_metricas(valor1, início_idx, fim_idx)
rms2, energia2 = calcular_metricas(valor2, início_idx, fim_idx)

# ---- 6. Função para plotar o sinal ----
def plotar_sinal(tempo, sinal, li, li2, rms, energia, titulo, cor, nome_arquivo):
    plt.figure(figsize=(10, 8))
    plt.plot(tempo, sinal, color=cor, label=titulo)

    # Linhas verticais
    plt.axvline(tempo[li], color='r', linestyle='--', label=f'início ({sinal[li]})')
    plt.axvline(tempo[li2], color='r', linestyle='--', label=f'fim ({sinal[li2]})')

    # Anotações dos tempos
    plt.annotate('Início: %f' % tempo[li], xy=(tempo[li], sinal[li]),
                xytext=(tempo[li], 4000),
                arrowprops=dict(facecolor='r', color='r'), color='r')
    plt.annotate('Fim: %f' % tempo[li2], xy=(tempo[li2], sinal[li2]),
                xytext=(tempo[li2], 4000),
                arrowprops=dict(facecolor='r', color='r'), color='r')

    # Tabela de métricas
    tabela = [[f'W = {rms:.2f}'],
              [f'Energia Média = {energia:.2f}']]
    plt.table(cellText=tabela, loc='upper left', collocs=(0, 1), celltextlines=(1, 1))

    plt.title(titulo)
    plt.xlabel('tempo')
    plt.ylabel('amplitude')
    plt.grid(True)
    plt.legend()

    plt.savefig(nome_arquivo, dpi=300, bbox_inches='tight')
    plt.close()

# ---- 7. Plotar o sinal ----
plotar_sinal(tempo, valor1, início_idx, fim_idx, rms1, energia1, 'Sinal 1', 'r', 'sinal1.png')
plotar_sinal(tempo, valor2, início_idx, fim_idx, rms2, energia2, 'Sinal 2', 'b', 'sinal2.png')

# ---- 8. Salvar CSV com métricas ----
metricas = pd.DataFrame({
    'canal': ['Valor1', 'Valor2'],
    'rms': [rms1, rms2],
    'energia_media': [energia1, energia2]
})
metricas.to_csv('metricas.csv', index=False)
```

O código agora **completa o ciclo de análise**: leitura → intervalo → cálculo → visualização → exportação.

As anotações e tabela tornam o gráfico **mais informativo e profissional**, ideal para relatório.

Mantém compatibilidade com o Google Colab (upload interativo e geração automática de arquivos).

Pontos ajustados após análise:

Inserir `plt.ylim(0, 4000)` garante uniformidade visual, mas pode precisar ser ajustado se outros sinais tiverem amplitude menor.

Seria interessante **automatizar nomes de arquivos** com base no CSV original, ex: "sinal1_<nome_do_csv>.png".

Também é possível integrar a **filtragem de média móvel** anterior nessa etapa para limpar o sinal antes da análise.

Aceitaamos a **Versão 3 (Visualização Completa)** como base consolidada.

Próxima meta: integrar a filtragem (média móvel) e permitir múltiplos uploads em sequência (versão final).

Objetivo: criar a versão final e consolidada do código, que realiza todo o fluxo de análise de sinais EMG: upload → filtragem → seleção manual de intervalo → cálculo de métricas (RMS e energia média) → geração de gráficos → exportação dos resultados.

Essa versão deve ser **robusta, interativa e pronta para relatório técnico**.

*Quero agora integrar todas as funcionalidades desenvolvidas nas versões anteriores em um único código completo.

O código deve:

- "Permitir upload de um ou mais arquivos CSV no Google Colab;"
- "Solicitar ao usuário o tempo de início e fim para análise;"
- "Aplicar automaticamente um filtro de média móvel (janela = 10) para suavizar os sinais;"
- "Calcular RMS e Energia Média para cada canal ('Valor1' e 'Valor2');"
- "Gerar dois gráficos por arquivo, com:"
 - "Linhas verticais marcando o início e o fim;"
 - "Tabela com métricas no canto superior esquerdo;"
 - "Eixo Y fixado de 0 a 4000;"
- "Salvar os gráficos e um CSV com métricas nomeado conforme o arquivo de entrada;"
- "Ao final, mostrar a lista dos arquivos gerados."

Use tudo o que já foi construído anteriormente como base, organizando o código de forma limpa, com funções modulares, docstrings e comentários explicativos."

```
def main():
    # Inicialização de variáveis
    inicio_tempo = float(input("Digite o tempo do início do sinal: "))
    fim_tempo = float(input("Digite o tempo do fim do sinal: "))

    # ... 2. Upload do(s) arquivo(s) CSV ...
    print("Faça o upload do(s) arquivo(s) CSV para análise.")
    uploads = files.upload()

    # Pegar o primeiro arquivo enviado
    arquivo = list(uploads.keys())[0]
    df = pd.read_csv(arquivo)

    # ... 3. Considerar colunas do CSV: tempo, valor1, valor2 ...
    tempo = df.iloc[:, 0].values
    valor1 = df.iloc[:, 1].values
    valor2 = df.iloc[:, 2].values

    # ... 4. Encontrar índices correspondentes ao intervalo manual ...
    inicio_idx = np.argmin(np.abs(tempo - inicio_tempo))
    fim_idx = np.argmin(np.abs(tempo - fim_tempo))

    # ... 5. Função para calcular RMS e energia média ...
    def calcular_metricas(tempo, inicio_idx, fim_idx):
        trecho = tempo[inicio_idx:fim_idx]
        rms = np.sqrt(np.mean(trecho**2)) # RMS correta
        energia_medio = np.mean(trecho)
        return rms, energia_medio

    rms1, energia1 = calcular_metricas(valor1, inicio_idx, fim_idx)
    rms2, energia2 = calcular_metricas(valor2, inicio_idx, fim_idx)

    # ... 6. Função para plotar e salvar sinal ...
    def plotar_sinal_manual(tempo, sinal, inicio_idx, fim_idx, rms, energia, titulo, cor='b', filename='grafico.png'):
        plt.figure(figsize=(12,8))
        plt.plot(tempo, sinal, label=titulo, color=cor)

        # Linhas verticais de início e fim
        plt.axvline(tempo[inicio_idx], color='r', linestyle='--', label='Início')
        plt.axvline(tempo[fim_idx], color='g', linestyle='--', label='Fim')

        # Anotar valores de início e fim no gráfico
        plt.annotate("({} Inicio: {:.2f})".format(sinal[inicio_idx], tempo[inicio_idx]),
                    xytext=(tempo[inicio_idx], sinal[inicio_idx]),
                    xy=(tempo[inicio_idx], 0),
                    arrowprops=dict(arrowstyle='->', color='red', color='red'))
        plt.annotate("({} Fim: {:.2f})".format(sinal[fim_idx], tempo[fim_idx]),
                    xytext=(tempo[fim_idx], sinal[fim_idx]),
                    xy=(tempo[fim_idx], 0),
                    arrowprops=dict(arrowstyle='->', color='green', color='green'))

        # Tabela no canto superior esquerdo
        tabela_valores = [{"x":0, "y":rms1, "text":f"RMS: {rms1:.2f}"}, {"x":0, "y":energia1, "text":f"Energia Média: {energia1:.2f}"}]
        plt.table(cellText=tabela_valores, loc='upper left', cellLoc='center', colwidths=[0.2,0.2])

        # Fixar o eixo Y de 0 a 4000
        plt.ylim(0, 4000)

        plt.title(titulo)
        plt.xlabel("tempo")
        plt.ylabel("Amplitude")
        plt.legend()
        plt.grid(True)

        # Salvar gráfico
        plt.savefig(filename, dpi=300, bbox_inches='tight')
        plt.close()

    # ... 7. Plotar e salvar os dois sinais ...
    plotar_sinal_manual(tempo, valor1, inicio_idx, fim_idx, rms1, energia1, "Sinal 1", cor='b', filename="sinal1.png")
    plotar_sinal_manual(tempo, valor2, inicio_idx, fim_idx, rms2, energia2, "Sinal 2", cor='g', filename="sinal2.png")

    # ... 8. Criar CSV com métricas ...
    dados_metricas = {
        "canal": ["Valor1", "Valor2"],
        "rms": [rms1, rms2],
        "energia_medio": [energia1, energia2]
    }
    df_metricas = pd.DataFrame(dados_metricas)
    df_metricas.to_csv("metricas.csv", index=False)

    print("📁 Arquivos gerados: sinal1.png, sinal2.png e metricas.csv")

if __name__ == "__main__":
    main()
```

O código é **modular**, com funções bem separadas (filtragem, métricas, plotagem e execução principal).

A **integração total** foi alcançada: agora o sistema executa todo o processo automaticamente.

É **escalável**: pode analisar múltiplos arquivos CSV em uma só execução.

Mantém compatibilidade com o **Google Colab** e saída organizada para relatórios.

Poderia incluir uma função de normalização para comparar sinais em escalas diferentes.

Futuramente, pode-se gerar um PDF consolidado com os gráficos e métricas.

Incluir barra de progresso (tqdm) para análise de muitos arquivos.

“Quero evoluir o código anterior de filtragem EMG para algo mais automatizado.

O novo código deve:

- “Pedir o nome do movimento (ex.: Abrir_Mao, Fechar_Mao, Flexao, Extensao);”
- “Permitir o upload de vários arquivos CSV pelo Google Colab;”
- “Criar automaticamente uma estrutura de pastas organizada por movimento e por janela (5, 10, 20);”
- “Aplicar o filtro de média móvel em cada arquivo e salvar CSVs filtrados;”
- “Gerar gráficos comparando o sinal original e o filtrado (para os dois canais Valor1 e Valor2) com escala fixa no eixo X e Y;”
- “Salvar todas as imagens PNG em suas pastas correspondentes;”
- “Compactar todas as pastas e arquivos gerados em um único ZIP para download automático.”

O código deve ser limpo, comentado e funcionar diretamente no Colab, sem necessidade de ajustes manuais.”

```
python
import os
import zipfile
import pandas as pd
import matplotlib.pyplot as plt
from google.colab import files

# Pergunta o nome do movimento
movimento = input("Digite o nome do movimento (ex.: Abrir_Mao, Fechar_Mao, Flexao, Extensao): ")

# Upload dos arquivos CSV
uploaded = files.upload()

# Pastas de saída
pasta_base = "/content/Dados Filtrados"
pasta_csv_base = os.path.join(pasta_base, movimento, "Csv Filtrados")
pasta_graficos_base = os.path.join(pasta_base, movimento, "Graficos")

# Cria pastas de janelas
janelas = [5, 10, 20]
for janela in janelas:
    os.makedirs(os.path.join(pasta_csv_base, f"janela_{janela}"), exist_ok=True)
    os.makedirs(os.path.join(pasta_graficos_base, f"janela_{janela}"), exist_ok=True)

# Função para filtrar e salvar
def aplicar_filtro_e_salvar(nome_arquivo, janela):
    dados = pd.read_csv(nome_arquivo)

    if "tempo (ms)" in dados.columns:
        tempo_col = "tempo (ms)"
    elif "tempo" in dados.columns:
        tempo_col = "tempo"
    else:
        raise KeyError("Coluna de tempo não encontrada (esperado: 'tempo (ms)' ou 'tempo')")

    dados["Valor1 Filtrado"] = dados["Valor1"].rolling(window=janela, center=True, min_periods=1)
    dados["Valor2 Filtrado"] = dados["Valor2"].rolling(window=janela, center=True, min_periods=1)

    nome_base = os.path.splitext(os.path.basename(nome_arquivo))[0]
    caminho_csv_filtrado = os.path.join(pasta_csv_base, f"janela_{janela}", f"{nome_base}_filtrado")
    dados_filtrados = dados[[tempo_col, "Valor1 Filtrado", "Valor2 Filtrado"]]
    dados_filtrados.to_csv(caminho_csv_filtrado, index=False)

    plt.figure(figsize=(10, 6))

    # Sinal 1
    plt.subplot(2, 1, 1)
    plt.plot(dados[tempo_col], dados["Valor1"], label="original", alpha=0.5)
    plt.plot(dados[tempo_col], dados["Valor1 Filtrado"], label=f"Filtrado (janela)", color="blue")
    plt.title('Sinal 1')
    plt.ylabel('Amplitude')
    plt.ylim(0, 4000)
    plt.xlim(0, 2000)
    plt.legend()
```

Essa versão consolidou o projeto em uma ferramenta **totalmente funcional e reproduzível**, permitindo processar grandes lotes de sinais EMG com apenas um comando.

A automação das pastas, gráficos e compactação marca uma **mudança de protótipo para sistema semi-final**, pronto para integração de análises quantitativas.

Justificativa do porquê aceitar ou rejeitar as sugestões da IA.

As sugestões apresentadas pela IA foram **majoritariamente aceitas**, pois mostraram-se adequadas e coerentes com os objetivos do projeto.

O código gerado atendeu completamente à necessidade de **automatizar o processamento de múltiplos sinais**, **organizar as saídas por janelas de filtragem** e **padronizar a geração dos gráficos**.

A decisão de **aceitar integralmente a proposta da IA** baseou-se nos seguintes pontos:

- A estruturação automática das pastas e do arquivo .zip otimizou o fluxo de trabalho, reduzindo etapas manuais e possíveis erros.
- O uso de **escala fixa nos gráficos** e a separação entre os sinais (Valor1 e Valor2) contribuíram para a padronização da análise visual.
- O código apresentou **clareza, boa legibilidade e comentários explicativos**, mantendo o padrão de reprodutibilidade necessário para relatórios técnicos.

No entanto, **algumas limitações justificam a rejeição parcial** de certas ideias implícitas da IA, como:

- A ausência de cálculos quantitativos (RMS e energia) limita a análise a aspectos puramente visuais;
- O sistema de geração de gráficos poderia incluir legendas automáticas com o nome do movimento e o arquivo analisado;
- Não há controle automático de normalização de sinais, o que reduz a precisão na comparação entre janelas diferentes.

Reflexão sobre a utilidade e as limitações da ferramenta no processo:

O uso da Inteligência Artificial foi **altamente útil** para acelerar o desenvolvimento do projeto.

Com poucos comandos, a IA foi capaz de gerar uma solução funcional e organizada, economizando tempo na escrita e depuração do código.

Além disso, a ferramenta facilitou a documentação do processo, permitindo registrar **cada iteração como uma etapa evolutiva do raciocínio computacional**.

Entretanto, a experiência também evidenciou **limitações importantes**:

1. **Ausência de contexto experimental** — a IA não compreende totalmente a natureza dos sinais EMG nem o significado físico dos parâmetros envolvidos, o que exige supervisão humana.
2. **Dependência de ajustes manuais** — embora o código execute corretamente, decisões como escala dos eixos, interpretação dos dados e avaliação da qualidade do filtro ainda precisam de análise técnica.
3. **Risco de superconfiança** — é necessário validar manualmente os resultados para evitar erros sutis de indexação, interpretação ou inconsistência numérica. Em resumo, a IA mostrou-se **uma ferramenta de apoio poderosa**, mas não substitui o papel do pesquisador. Sua principal contribuição foi **otimizar o tempo de implementação**, permitindo concentrar esforços na **interpretação científica dos resultados e na evolução metodológica do projeto**.

6. Reflexão e aprendizado

O que aprendi sobre análise de dados

Durante o desenvolvimento deste projeto, aprendi na prática a importância da **preparação e limpeza dos dados** antes de qualquer interpretação. A etapa de análise de sinais EMG mostrou

que dados brutos podem conter **ruído, saturação e inconsistências**, e que técnicas como a **filtragem por média móvel**, cálculo de **média aritmética** e **RMS** são fundamentais para tornar o sinal mais confiável.

Compreendi também que a **visualização gráfica** (como os gráficos de dispersão e comparativos entre sensores) é uma ferramenta poderosa para revelar padrões que não seriam percebidos apenas por análise numérica. Assim, percebi que a análise de dados é um processo iterativo, que exige observar, testar, filtrar e validar constantemente as informações.

O que aprendi sobre colaboração com IA

A interação com ferramentas de Inteligência Artificial, como o ChatGPT, foi essencial em diversas etapas do projeto. Aprendi a utilizar a IA de forma crítica e colaborativa, buscando auxílio na explicação de conceitos, na geração de códigos em Python e na interpretação dos resultados, mas sempre revisando e ajustando conforme as necessidades do projeto.

Percebi que a IA não substitui o raciocínio humano, mas amplia a capacidade de análise, permitindo resolver problemas técnicos de forma mais rápida e eficiente. Essa colaboração ajudou a transformar ideias em resultados práticos e organizados, além de otimizar a documentação do trabalho.

Dificuldades encontradas e como foram superadas

A principal dificuldade foi lidar com o nível de ruído e saturação dos sinais coletados pelo ESP32, que inicialmente distorciam os resultados e dificultavam a identificação dos movimentos. Esse problema foi superado com a inserção de um divisor de tensão no circuito de aquisição, que reduziu a amplitude do sinal, e com a aplicação dos filtros de média móvel, que suavizaram o ruído.

Outra dificuldade foi o posicionamento dos sensores EMG, que exigiu vários testes até encontrar pontos que gerassem respostas consistentes. Por fim, houve também o desafio de interpretar corretamente os gráficos e métricas, que foi superado com o apoio da IA e a prática de repetidas análises comparativas.

Possíveis melhorias para um próximo projeto

Para as próximas etapas, é possível implementar **filtros digitais mais avançados**, como filtros passa-baixa de Butterworth ou filtros adaptativos, que preservam ainda mais a forma do sinal.

Também seria interessante utilizar **um módulo de aquisição dedicado a EMG** (com ADC de maior resolução e taxa de amostragem constante), garantindo medições mais precisas.

Outra melhoria seria aumentar o **número de coletas e participantes**, para enriquecer o dataset e torná-lo mais robusto para futuras aplicações de **redes neurais** e **classificação automática de movimentos**.

Além disso, planeja-se integrar o sistema a uma **interface de controle em tempo real**, simulando a movimentação de uma prótese, consolidando assim a ligação entre o estudo teórico e sua aplicação prática.

7. Conclusão

O desenvolvimento deste projeto permitiu compreender, de forma prática e detalhada, como técnicas de **análise de dados** e **processamento de sinais** podem ser aplicadas para a identificação e diferenciação de **movimentos musculares** a partir de sinais mioelétricos (EMG).

Através da coleta real de dados utilizando **sensores EMG conectados ao microcontrolador ESP32**, foi possível observar as principais características dos sinais elétricos produzidos pelos músculos durante os movimentos de **flexão, extensão, abrir e fechar o punho**.

A aplicação do **filtro de média móvel** (com janelas de 5, 10 e 20) demonstrou ser eficiente para **reduzir ruídos e suavizar as oscilações**, resultando em sinais mais limpos e adequados para análise. Entre as janelas testadas, a de **10 amostras** apresentou o melhor equilíbrio entre suavização e preservação das informações fisiológicas.

O cálculo da **média aritmética** e da **média RMS** possibilitou quantificar a intensidade dos sinais e comparar os diferentes movimentos de forma estatística. Essas métricas, quando organizadas em um **gráfico de dispersão (scatter plot)**, evidenciaram **diferenças claras entre os padrões musculares de cada movimento**, comprovando a eficácia da metodologia adotada.

Em relação aos **objetivos iniciais**, que envolviam **processar, filtrar e analisar sinais EMG** para identificar padrões característicos e distinguir diferentes tipos de movimentos, todos foram atingidos com êxito. O projeto demonstrou que é possível, com recursos acessíveis e técnicas adequadas de processamento, **reconhecer movimentos musculares específicos** e gerar informações consistentes para aplicações futuras no **controle de próteses biônicas**.

Por fim, o trabalho reforçou a importância da análise de dados na engenharia biomédica e mostrou como a **colaboração entre métodos experimentais, processamento digital e apoio de Inteligência Artificial** pode transformar dados brutos em **conhecimento aplicável e resultados funcionais**, servindo como base para o avanço de sistemas de controle muscular inteligentes.

8 – Referencias

Fonte do Dataset:

Os dados utilizados neste projeto foram coletados experimentalmente pelo próprio grupo de pesquisa, a partir de medições reais de sinais mioelétricos (EMG) obtidos por sensores de superfície conectados a um microcontrolador **ESP32**. Os sinais foram adquiridos com eletrodos ECG descartáveis posicionados sobre os músculos do antebraço, gerando arquivos **.CSV** contendo as amostras de tempo e amplitude elétrica para diferentes movimentos musculares (flexão, extensão, abrir e fechar o punho).

Ferramentas de Desenvolvimento e Análise de Dados:

- **Python 3.11** – Linguagem principal utilizada no processamento e análise dos dados.
- **Pandas** – Para leitura, organização, limpeza e manipulação dos arquivos CSV.
- **NumPy** – Para cálculos matemáticos e estatísticos (média, RMS, energia média).
- **Matplotlib** – Para geração dos gráficos dos sinais, comparações e dispersões.
- **Seaborn** – Para visualização complementar e estilização gráfica.
- **Jupyter Notebook / Google Colab** – Ambiente de execução dos códigos e visualização dos resultados.
- **ChatGPT (OpenAI)** – Ferramenta de apoio utilizada para explicação de conceitos teóricos, estruturação do relatório e auxílio na documentação dos códigos.
- **Microsoft Excel** – Utilizado para conferência e organização dos dados resultantes das médias RMS e aritméticas.

Referências Complementares:

- DE LUCA, Carlo J. *The Use of Surface Electromyography in Biomechanics*. Journal of Applied Biomechanics, v. 13, n. 2, p. 135–163, 1997.
- MERLETTI, R.; PARKER, P. A. *Electromyography: Physiology, Engineering, and Non-Invasive Applications*. Wiley-IEEE Press, 2004.
- Documentation: *Pandas, NumPy, Matplotlib e Seaborn Libraries*. Disponível em: <https://pandas.pydata.org>, <https://numpy.org>, <https://matplotlib.org>, <https://seaborn.pydata.org>.
- OpenAI. *ChatGPT – Assistente de Inteligência Artificial*. Disponível em: <https://chat.openai.com>.